Photometry Roadmap

William Fong edited this page yesterday · 10 revisions

This document outlines the current process of creating QLP light curves on the pdo compute system. The process starts when TICA calibrated FFIs for an orbit are delivered to `/pdo/qlp-data/tica-delivery/sXXXX`, and finishes with fully detrended and quality-flagged light curves for the orbit. After completing these steps for the final orbit of a sector, operators should head to the Planet Search Roadmap.

Summary: Just the commands

This sections lists the command templates used in order for experienced operators. If this is your first time operating, or if you run into problems, find more details in subsequent sections.

> Tip
>
> Ideally, run these commands on `pdogpu{4,5,6}` (a machine with A30 GPUs) to accelerate the ePSF fits!

> Warning
>
> There is a current bug where ingesting frames while a *partial* TICA delivery has been made will result in a broken orbit. Ingest frames only while *complete* FFI deliveries have been made. Attempts will be made to mitigate this as much as possible.

```
cd /pdo/qlp-data
./new-qlp-orbit.sh $ORBIT $SECTOR
cd /pdo/qlp-data/orbit-$ORBIT/ffi/run
tglc all --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 80
qlp lcdb ingest ffi -n 50 --sector $SECTOR /pdo/qlp-data/tica-delivery/s0$SECTOR
qlp lcdb partition sync
qlp patools qflag -c qlp.cfg --cams-ccds $CAM
qlp lctools detrend -c qlp.cfg --source h5 --autolist $CAM -b -n 80
qlp lctools qualityflag -c qlp.cfg --autolist $CAM -n 80
qflag-plot
# Locally: ssh -L 5000:localhost:5000 pdoX
# Open http://localhost:5000/ to inspect quality flags
sed -i '$START,$END s/ 0/ 1/g' cam${CAM}ccd${CCD}_qflag.txt
qlp lcdb ingest lightcurve -n 80 /pdo/qlp-data/orbit-$ORBIT/ffi/cam{1..4}/ccd{1..4}/LC
qlp lcdb ingest qflag --orbit $ORBIT --camera $CAM --ccd $CCD /pdo/qlp-data/orbit-${ORBIT}/ffi/run
```

Orbit setup

A script in `/pdo/qlp-data/` sets up a new orbit directory with proper permissions and configuration.

```
cd /pdo/qlp-data
./new-qlp-orbit.sh $ORBIT $SECTOR
```

> Tip
>
> Running the script with no arguments shows the proper usage, if you ever forget.

Extracting light curves with TGLC

TGLC provides a command line interface. In most cases, only a single command will be needed, which sequentially executes each subcommand.

> Tip
>
> TGLC should ideally be run on a GPU machine to enable GPU acceleration of the core ePSF fitting. A30 GPUs (pdogpu 4,5,6) have demonstrated by far the best performance. Running TGLC on a non-GPU machine will use the CPU for ePSF fits.

```
cd /pdo/qlp-data/orbit-$ORBIT/ffi/run
tglc all --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 80
```

If you are on a GPU machine, the number of workers for the ePSF step will be adjusted to the number of GPU devices available to avoid oversubscribing them.

If there is an error during one of the steps (it should be clear which one due to the headers logged by the `all` command), you can restarted from the step that failed using the subcommands below. All commands try to avoid repeating work, so rerunning the `all` command would skip many unnecessary steps if rerun (though it is still faster to rerun only the steps that need it).

```
tglc catalogs --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 80
tglc cutouts --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 80
tglc epsfs --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 8
tglc lightcurves --tglc-data-dir /pdo/qlp-data -o $ORBIT --ccd 1,1 1,2 1,3 1,4 -n 80
```

Use the `-h` option with any TGLC command to show the command line arguments available. Some arguments are command to all commands, and are listed below.

```
-o ORBIT, --orbit ORBIT
    TESS orbit to run
--ccd CCD [CCD ...]
    cam,ccd pairs to run. For example, --ccd 2,4 specifies camera 2, CCD 4.
    Multiple arguments are allowed separated by spaces, like --ccd 2,4 3,2.
--cutout CUTOUT [CUTOUT ...]
    x,y pairs of cutouts to run. Coordinates are among all cutouts, so
    --cutout 0,0 specifies the bottom left cutout and --cutout 13,13 specifies
    the top right cutout (with default size). Multiple argumenst are allowed
    separated by spaces, like --cutout 1,1 1,2
-n NPROCS, --nprocs NPROCS
    Number of processes to use
-r, --replace
    Whether to overwrite existing data products
--tglc-data-dir TGLC_DATA_DIR
    Base directory for TGLC data. Default is first directory containing the
    working directory called "tglc-data", or the working directory if no
    "tglc-data" directory is found.
--debug
    Output debug-level logs (default=info-level logs)
-l LOGFILE, --logfile LOGFILE
    File to write logs
--enable-runtime-warnings
    Allow numpy runtime warnings (silenced by default)
```

Post-processing light curves with QLP

After TGLC extracts the light curves, QLP detrends them and adds CCD-wide quality flags. Before detrending, binned quaternions need to be gathered (`patools qflag`) and the header data from the FFIs needs to be ingested into the light curve database.

```
qlp lcdb ingest ffi --sector $SECTOR -n 50 /pdo/qlp-data/tica-delivery/s00$SECTOR
qlp patools qflag -c qlp.cfg --cams-ccds $CAM
qlp lctools detrend -c qlp.cfg --source h5 --autolist $CAM -b -n 80
qlp lctools qualityflag -c qlp.cfg --autolist $cam -n 80
```

Below are the most important options for these commands:

- `detrend`
  - `--autolist` specifies which cams/ccds to run on. Using just the camera number (e.g., --autolist 1) will run on all ccds for that camera. To run only on certain ccds, list the ccds after a colon: --autolist 1:2,3 will run on camera 1, ccds 2 and 3. Specify multiple cameras (and corresponding ccds) by listing with spaces in between: --autolist 1:2,3 4 will run on camera 1, ccds 2 and 3; and camera 4, all ccds. This command can also take an --inlist (list of TICs) or --tic (specific TIC IDs, space-separated if there are multiple) instead of --autolist.
  - `--bkspace-min` can be overriden at the command line.
  - `-b` indicates that this detrending method should be saved as the best detrending method in each h5 file. If this flag is not given, the detrended magnitude and error will be added to the file, but they will not be set as the "best" detrending method.
- `qualityflag`
  - If the generated flags are all ones, try adding `--rms $rmsfactor` to the end with different values of `$rmsfactor` (`1.5`, `2.0`, or `3.0`) until the flags look correct.

Once the quality flags are generated, they need to manually inspected and modified. First, start the quality flag plotter server on pdo.

```
qflag-plot
```

Next, forward port `5000` from the machine running the server to your local machine:

```
ssh -L 5000:localhost:5000 pdoX
```

Note that pdoX should be replaced with a specific pdo machine name (e.g., pdo8).

Now, open your browser to http://localhost:5000/ and view the light curves and their quality flags using the Light curve plotter. Adjust the quality flag files in the run directory (e.g., `camXccdY_qflag.txt`) as necessary to mask out LC systematics (e.g., momentum dumps, scattered light, etc.). You can use `sed` to change all 0s to 1s in a range of lines.

```
sed -i '$START,$END s/ 0/ 1/g' cam${CAM}ccd${CCD}_qflag.txt
```

Once you are happy with how the quality flags look, ask another operator to verify that they look good and that the light curves are correctly detrended.

Ingest light curves into the database

Finally, the light curves are ready to be ingested using `lightcurvedb`.

```
qlp lcdb ingest lightcurve -n 80 /pdo/qlp-data/orbit-$ORBIT/ffi/cam{1..4}/ccd{1..4}/LC
```

If you run into problems, try `lcdb --logging debug lightcurve ...` for more detailed output. The database has 500 available connections, but too many connections can cause bottlenecks, e.g. with write speed on pdodb. Aim for roughly 250-300 total connections between all ingests at a given time. If you're using non-gpu machines, you can probably use the normal number of processes given the number of cores on the machine. If you're using a pdogpu machine, limiting to 80 cores per camera is a good idea.

To check that the ingestion completed ok, use the script `count_lcs_db.py` which lists the number of light curves per CCD ingested in the database, and compare to the number of light curves in the LC dirs. Note that `count_lcs_db.py` can take well over an hour to run.

```
python3 count_lcs_db.py $ORBIT
```

> Tip
>
> The `ingest-dir` command works via filepaths, so you can pass wildcards or any bash-resolving patterns. The above example ingests all four cameras and ccds, but you may want to ingest each cam/ccd one at a time instead.

HLSP production

If it is the final orbit of a sector, HLSP light curves for the MAST repository need to be created. There is a QLP command that converts the H5 light curves to the FITS format expected by MAST. Create a directory somewhere to house the FITS light curves (a scratch disk is often a good idea for good performance, especially since this directory will not be kept long-term). Make sure the proper direnv configuration is linked there, and then run the QLP conversion command.

```
cd /scratch/hlsp
mkdir s00$SECTOR
ln -s /sw/qlp-environment/qlp-master-envrc .envrc
direnv allow
qlp lctools hlsp -o s00$SECTOR -a $CAM -n 72
```

Some notes:

- This command creates a series of subdirectories based on the 16-integer TIC ID for each star. E.g., TIC 44300180 will have a .fits file created in the subdirectory `/0000/0000/4430/0180/`.
- This command will do a file verification step at the end to check for empty/corrupted files. Make sure no files have errors before proceeding. If files need to be remade, you can use `-i $inlist` (instead of `-a $CAM`) pointing to a list of TICs.

To package and deliver the HLSPs, tar them, move them to `/pdo/qlp-data/FITSLC/`, and create a checksum file.

```
tar -czvf hlsp_qlp_tess_ffi_s00${SECTOR}_tess_v01_bulk-dl.tar.gz s00$SECTOR
mv hlsp_qlp_tess_ffi_s00XX_tess_v01_bulk-dl.tar.gz /pdo/qlp-data/FITSLC
cd /pdo/qlp-data/FITSLC
sha256sum hlsp_qlp_tess_ffi_s00${SECTOR}_tess_v01_bulk-dl.tar.gz > hlsp_qlp_tess_ffi_s00${SECTOR}_tess_v0
```

Finally, notify a MAST representative that the HLSPs are ready.

> Important
>
> **Troubleshooting:** If you get a "File I/O error", trying the following environment variable export. This is a known issue, but the root cause of the problem is unclear.
>
> ```
> export HDF5_USE_FILE_LOCKING=FALSE
> ```

Next steps

If the orbit is not the final orbit of the sector, you are done! If it is the final orbit of the sector, head over to the Planet Search Roadmap and follow the steps from there to run the sector processing.

Pages 13

Use the "pages" section of the sidebar to see a table of contents for each page, including the current page.

For Operators

- New Operator Setup
- Photometry Roadmap
- Planet Search Roadmap
- TEV Delivery
- Running QLP Branches
- Planet Search in Ecliptic Sectors

For QLP Admins

- QLP Operating Environment
- Releasing and Deploying QLP
- Managing QLP Branches
- Managing and Deploying Astronet
- Updating the TESS Ephemeris

Clone this wiki locally

`https://github.com/mit-kavli-ins`

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information